

Lecture 4: Distributed Hash Tables (part 1)

Christoffer Fink

September 9, 2026

This is the “handout” (or lecture notes) version of the (part 1) lecture on DHTs. Note that each page shows two presentation slides on the left, with notes (like this one) on the right.

This lecture

- DHTs
  - Multiple example DHTs
  - DHTs vs Overlay networks
- Goal: try to design traditional (in-memory) data structures that we can distribute
- Reinvent a few example DHTs from scratch
- Motivating use case (requirements/restrictions)
- Understand design constraints rather than jumping to the answer at the back of the book

Note: Kademlia will be covered in detail in part 2. (**Recorded only.** You are welcome to ask questions in the lab sessions.)

In this lecture, we’ll look at distributed hash tables (DHTs) by taking regular data structures as a starting point. As far as I know, this is an unusual approach to teaching the subject. So let me briefly motivate it.

First, you should already have some familiarity with the data structures we’ll consider, which should make things feel concrete. Second, I believe the attempt to take a regular, in-memory data structure and simply trying to make a distributed version of it, by moving different parts of it to separate machines, illustrates very clearly what the constraints are and (some of the reasons) why we have to think about distributed systems differently. Third, by trying out a few “obvious” but naive designs and seeing exactly why they fail, you’ll have a much better understanding of ***why*** *other designs* ***do*** *work*, i.e. what challenges they are designed to overcome. That is, the dead ends are as instructive as the final answer. Fourth, by “deriving” the designs from scratch you will likely remember how they work better than if I told you about them directly, as if the solution was handed down on stone tablets.

I should say a few more things about that last point. I’m trying to show you a plausible line of reasoning that you might have followed to re-invent/discover these DHT designs on your own. It’s not a true derivation in the sense that they are logically necessary consequences, but they often follow naturally from a few reasonable (often simplifying) assumptions. Also, while I am trying to lead you along a plausible path of discovery/invention, it is almost certainly not a historically

Goal

What we're trying to build

- First: Large scale
  - Lots of data, requests, and/or nodes.
- Later: Decentralized
  - No single point of failure.
  - Nobody "in charge".
- Fault-tolerance
  - Must handle churn (nodes come and go).

Let's imagine we want to build a simple Key-Value Store (KVS). First, we'll just worry about the scale. That is, we have to support a large amount of data and requests. So, we will start by thinking about a simple client-server model and how we might build a server that can handle many requests. Then we'll try to make it distributed, i.e. spreading the data structure out among multiple nodes. (This will reveal some challenges we have to work around.) And later we will consider how to also make it a decentralized system, where there are no special nodes and nobody is in charge.

That also means nodes can come and go at any time, which means we have to handle churn and fault-tolerance.

Simple Key-Value Store (KVS)

- We want a simple, centralized KVS (DB).
- Clients
  - can put key-value pairs
  - get values associated with keys
- How could we implement this KVS (inside)?
- What's the simplest solution?

Our KVS should support `put(key, value)` operations that allow clients to insert new key-value pairs, i.e. associating a key with some value. And it should support `get(key)` operations that allow clients to retrieve a value by its key, i.e. the value associated with the key.

Now, what's the simples data structure you can think of that would allow us to implement this KVS server?

Plain array

Simple Key-Value Store (KVS)

- How about a plain array storing key-value pairs?
- Need to search through the array for both `put` and `get` ($\Theta(n)$).
- Can efficiently grow the array ($\Theta(1)$ amortized). But need to check if already present.
  - Could theoretically `put` in $\Theta(1)$ (amortized) by simply appending to the end.
  - Worst case: can't find latest mapping; best case: duplicates. Either way: array full of garbage.
- OK for small arrays, but doesn't scale.
- Linked list would also be equally OK (or not).
  - Easy to grow by one node at a time.
  - Same $\Theta(n)$ problem for put/get.
- But unlike linked lists, arrays have $\Theta(1)$ random access. How to exploit?

The **simplest** solution I can come up with is just a plain array with key-value pairs. Then both `put` and `get` are linear, which is not great for handling large scale. We could theoretically blindly append a new key-value pair on `put` and search from the end so we find the most recent pair. Then `put` would be constant-time *amortized* (since we occasionally have to grow the array), but then it would fill up with garbage if we are overwriting keys with new values by just appending them. In fact, `get` would then no longer be linear in the number of key-value pairs we want to store, but instead linear in the number of pairs ever appended! So we end up with $\Theta(n)$ `put` and `get` because we need to do a linear scan through the array in both cases.

We get the same $\Theta(n)$ problem with a linked list. Although, we potentially have the advantage that adding a single KV pair is simpler.

Unlike a linked list, arrays allow $\Theta(1)$ random access. Maybe we can exploit that somehow?

Sorted array

Simple Key-Value Store (KVS)

- If we keep key-value pairs sorted by key, we can achieve $\Theta(\log_2 n)$ `get`.
- But `put` is still $\Theta(n)$.
- Because we need to shift over $\Theta(n)$ elements to make space!

If we maintain the array of key-value pairs sorted by key, we can use binary search and implement a logarithmic-time `get`. But then `put` is still linear. While we can use binary search to find where to insert the new KV pair, we then have to shift over $\Theta(n)$ items in order to make space for the insertion. OK, so we could achieve $\Theta(\log_2 n)$ `get` and $\Theta(n)$ `put`. Maybe that's OK if we have mostly reads and not many writes. But let's see if we can do better.

Can we exploit the $\Theta(1)$ random access even more?

Add buckets

Simple Key-Value Store (KVS)

- Even more direct access? Exploit random access even more?
- If only keys were indices, we could access the right key-value pair directly! ($\Theta(1)$)
- Converting keys to integers is trivial...
  - Had to be turned into a bit string to send over the network. Already an integer!
  - Can always hash. E.g. `Object.hashCode()` or `SHA-1`.
  - Can't use arbitrary-size integers, nor 160-bit integers!
  - For direct access (index) we need something we can allocate memory for!
- We would need to map them to a much smaller (and **finite**) range.
- But then, by the pigeon hole principle, some keys map to the same index!
- Let's divide the single large array into multiple smaller buckets.

If only the keys were indices instead of arbitrary values, then we could just access the right array element directly! Both `put` and `get` would be constant-time. Now, converting keys to integers is trivial. For one, they had to be turned into a string of bits so they could be sent over the network. So we could trivially interpret those bits as an integer. Also, we can always just hash the keys.

But just turning them into integers is not enough. If we turn arbitrary keys into bit strings, we get arbitrarily large integers. But we can't allocate an arbitrarily large array and access elements by an arbitrarily large index. While something like SHA-1 or SHA-256 produces integers in a finite range, it's still way too large, because we have nowhere near enough memory for even 2^{160} items.

Using the equivalent of `Object.hashCode()` might technically be barely feasible, but then the range is so small that collisions are highly likely (different keys mapping to the same index). Using a larger key space (like 160 or 256 bits) reduces the chance of collisions but is too large for direct indexing.

Add buckets

Simple Key-Value Store (KVS)

- Let's divide the single large array into multiple smaller buckets.
- Map keys to bucket index. (Possibly after first hashing)
- Each bucket i holds all key-value pairs whose keys map to index i.
- Add more buckets as needed to keep each bucket small ($c \approx n/m$, or $c \in \Theta(n/m)$ in each bucket).
- Can directly get to the right bucket ($\Theta(1)$)
- and then just search the (usually small) bucket (expected $\Theta(c) = \Theta(1)$).
- Both `put` and `get`: $\Theta(1) + \Theta(c) = \Theta(1)$.

What if we divide up the array into "buckets". So instead of a single large array of KV pairs, we'll have an array where each element i is an array holding all the KV pairs where the key maps to index i. When there are n KV pairs, we'll have m buckets. If we choose some small constant $c \in \Theta(n/m)$, we can add more buckets as needed to make sure each bucket has approximately c KV pairs. Then we can directly get to the right bucket in $\Theta(1)$ time and then search through that bucket in $\Theta(c) = \Theta(1)$ time.

Modulo hashing

Simple Key-Value Store (KVS)

- If we have m buckets, then we'd like $1/m$ of the keys/hashes to map to each bucket.
- So just put every m^{th} in bucket i.
- Modulus: $i = k\%m$
- Assuming keys/hashes are uniformly distributed, each bucket will have $c \approx n/m$ keys.

But how do we map a key (or its hash) to an index? If we have m buckets, we would like n/m of the keys/hashes to map to each bucket. In other words, we would like to put every m^{th} KV pair in bucket i. A simple way to achieve that is to compute the modulus (integer division remainder) of each key by the number of buckets. So key k would have index $i = k\%m$.

Assuming the keys/hashes are uniformly distributed, the indices will also be uniformly distributed, and each bucket will have $c \approx n/m$ keys **in expectation**. Note that this uniform distribution assumption means we only get **expected** $\Theta(1)$ `put` `get`, *not worst-case*. We are also trading some memory for time.

Add buckets

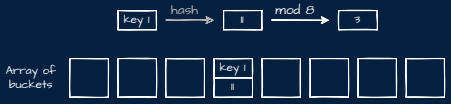
Simple Key-Value Store (KVS)



Here's our array of buckets.

Add buckets

Simple Key-Value Store (KVS)



In this example, `key 1` hashes to $k = 27$. With $m = 8$ buckets, the key maps to bucket $i = k\%m = 27\%8 = 3$.

Add buckets

Simple Key-Value Store (KVS)



Here we have a few more key-value pairs. Note that some buckets are empty while some have more than one pair.

Traditional Hash Table

Simple Key-Value Store (KVS)

- We've just reinvented the classic hash table data structure!
- **Expected** $\Theta(1)$ `put`/`get`.

The $\Theta(1)$ expected time relies on uniformly distributed keys. If we get unlucky with the keys, they could all map to the same bucket, and then we would have $\Theta(1)$ `put` and `get` (worst-case).

Scaling up

Distributed KVS

- What if the workload (number of requests or size of data) is too big for one server to handle?
- Replication + load balancer?
- Has limitations, especially if writes are common. Replicas need to be kept in sync/consistent.
- Didn't we just solve this?

What if the amount of data and/or the number of requests is just too big for a single server to handle? If we would replicate the KVS on multiple nodes, we could use a load balancer to distribute the work among the replicas. But that doesn't help with the amount of data. Also, it is much more straightforward if we have mostly reads. If writes are common, we may need to do extra work to keep the replicas synchronized so we have consistent data. (Although, this depends a lot on what kinds of consistency guarantees we actually need.)

Now, you might wonder if we didn't just solve this problem...

Scaling up

Distributed KVS

- We divided up a big array of key-value pairs into smaller buckets.
- What if we divide the KVS into smaller KVSs and **partition the keys** among them?
- In the database context, this is also called **sharding**.
- We are partitioning keys into different buckets in our hash table!
- Maybe we can just make a **distributed** version of our hash table, where different buckets are on different nodes?

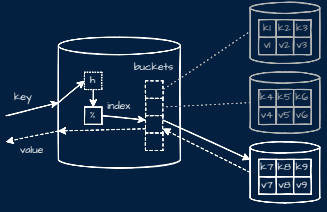
We essentially divided up a big array into smaller buckets and partitioned the keys among them. This is a common technique known as “sharding” in the database context. Each database is responsible for a subset of the keys/data. Each shard stores much less data, and load balancing follows automatically from the key partitioning because each shard only has to handle requests for its subset of the data.

So, maybe we can just “distribute” our hash table by putting each bucket on a separate node.

Distributed hash table?

Maybe?

- Let each bucket be a proxy for the node that stores the bucket contents.
- The bucket index identifies the node, and we ask it to `put`/`get`.
- Many problems...



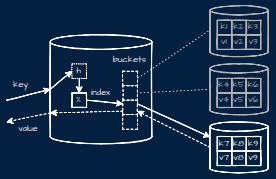
That is, each bucket is just a proxy for the node that is actually storing the KV pairs in that bucket. We find the node using the bucket index, and then we ask it to handle the `put` or `get` request. Internally, each bucket could then be a flat array of KV pairs, another hash table (possibly delegating to yet more buckets/nodes), or something else.

But this solution has several serious problems.

Get only

Distributed hash table?

- Kind of works if we mostly have `get` requests.
- We would have a central primary/root server that implements the KVS and delegates to nodes that store buckets.
- Essentially works as a (poor) load balancer.
- Root has a bit less work to do to find keys, but all traffic still goes through it.
- Bypassing root with caching is not feasible. (Key-to-index mapping is messy.)
- Big problems with `put`!



Again, this kind of works if we have mostly `get` requests. (As we'll see shortly, `put` is what really ruins everything.)

Note that we would have some sort of tree with a single, central root. That root node doesn't store any keys, but essentially works as a load balancer. While the root has a bit less work to do, all traffic still goes through it. It would be nice if we could bypass it by caching which bucket node clients should be talking to, but the problem is that the key-to-index mapping depends on the number of buckets. As the number of KV pairs increases, the number of buckets increases, and then the indexing changes. So keys can suddenly map to completely different buckets/nodes.

This remapping is a general problem that we have to solve somehow, because that is why `put` is such a problem.

Doesn't work with put

Distributed hash table?

May need to grow on `put`!

- Double the number of nodes (servers) each time? Dubious!
- Much more serious problem: Moving keys between buckets!
- When we change the number of buckets, we change the key-to-index mapping!

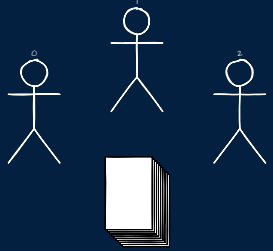
The problem is that `put` may require that we increase the number of buckets. We know how to do that so that we can keep `put` constant-time **amortized**. We just increase the number of buckets by a constant factor, such as doubling.

But doubling the number of buckets would mean we double the number of nodes! That's a bit questionable. Especially considering we may also need to shrink at some point, we would likely have many idle nodes allocated.

The even bigger problem is that, as the number of buckets changes, the key-to-index mapping changes. When we have a regular, in-memory data structure like a dynamic array or a hash table, copying elements from the old array to the new array is relatively cheap. We just need to copy some pointers in memory. But now moving a KV pair from one bucket to another means copying it from one machine to another, over the network. Let's take a closer look at this problem of **key movement**.

Card dealing analogy

Key movement

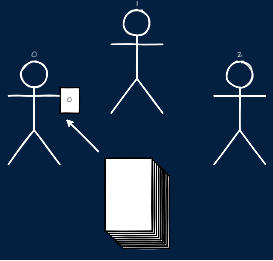


Let me illustrate the problem with a card-dealing analogy.

So, we have three players, and we'll deal the whole deck of cards to the players in the usual way.

Card dealing analogy

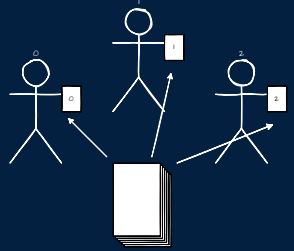
Key movement



That is, we give the first card to the first player...

Card dealing analogy

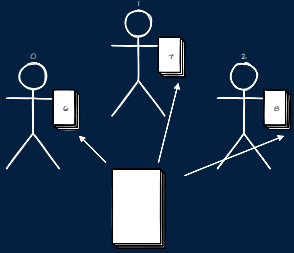
Key movement



...then the second card to the second player. And so on...

Card dealing analogy

Key movement

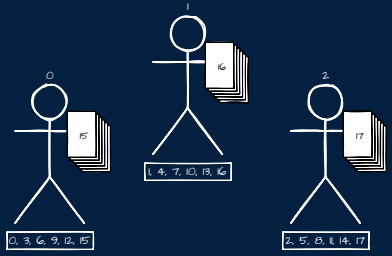


...then we go back to the first player and start over, cycling through all the players until the whole deck has been dealt.

Notice that this corresponds exactly to the "modulo hashing" we used to map keys to buckets! With m players, player i gets each card k such that $i = k\%m$.

Card dealing analogy

Key movement

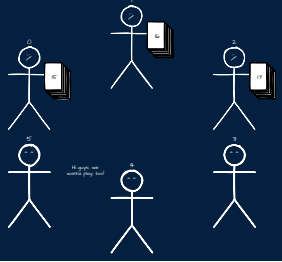


Now, I'm not quite sure what the rules of this game are, but the key point is that each player has to keep track of which player has which cards. So, for each card k, the players have to know which player i has it.

Card dealing analogy

Key movement

- Now three more players want to join.
- Need to redistribute cards to the new players, but **keep track of who has which cards!**



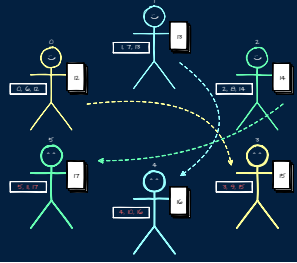
Now suppose three new players show up and want to join the game. They need some cards too, and it would be great if we could give them cards in such a way that we can still easily keep track of who has which cards.

Can you think of a way that we can redistribute the cards so that each player ends up with *the cards it would have had if they had been part of the game from the beginning?*

Card dealing analogy

Key movement

- Player 0 gives **every second card** to player $3 + 0$.
- Player 1 gives **every second card** to player $3 + 1$.
- Player 2 gives **every second card** to player $3 + 2$.



Actually, it's pretty easy. Note that the number of players doubled. So we can just do what we would do when doubling an array of buckets!

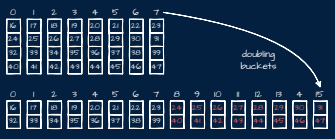
Player 0 ended up with the cards that both player 0 and player 3 would have gotten. (And player 1 ended up with the cards it and player 4 would have gotten, etc.) In particular, player 0 got cards 0, 3, 6, 9, 12, etc. And player 3 *would have gotten* cards 3, 9, 15, etc. Dealing would have cycled back to player 0 half as often, and each player would get half the cards. In particular, every second card was a card one of the three new players would have received if they had been there at the start.

So, each player i just needs to give every second card to player $3 + i$. That way, each player ends up with exactly the cards they would have received if they had started with 6 players instead of 3.

Card dealing analogy

Key movement

- Doubling from m to $2m$.
- Each player i gives **every other card** to player $m + i$.
- All players transfer half their cards. Total: $\Theta(n/2) = \Theta(n)$.
- Buckets: Moving $\Theta(n)$ keys over the network!
- So... maybe just add one bucket at a time? Already thought doubling was dubious anyway.



Let's generalize this a bit. Here's also an illustration in terms of arrays rather than cards and players.

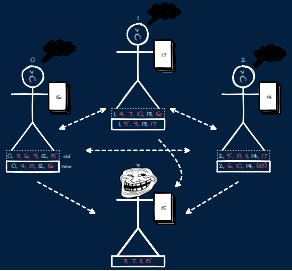
While this is a nice and elegant solution in the card dealing analogy, notice that each player has to hand over half of its cards. In total, half of all the cards/keys move! That is, if there are n KV pairs, $\Theta(n)$ KV pairs move from one bucket to another because the key changes its index to a new bucket. That is a linear amount of key movement, meaning that as the set of data grows, we need to move more and more data **over the network**. That's bad news!

Considering that doubling the number of nodes sounded questionable anyway, let's consider just adding a single new bucket at a time. We know that means `put` is $\Theta(n)$ rather than $\Theta(1)$ amortized, but at least that's in-memory copying within the node. So maybe that is an acceptable sacrifice if it means that we can avoid moving keys over the network. (Considering that copying memory is many orders of magnitude faster than copying over the network.)

Card dealing analogy

Key movement

- Now **one** new player joins.
- Need to deal over from scratch!

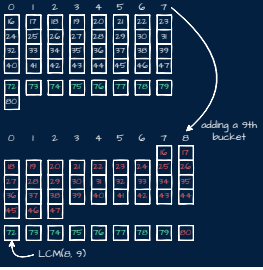


Going back to our card dealing analogy, now a single new player joins. As you can see, now essentially all the cards/keys need to move, and we might as well deal the cards from scratch.

Card dealing analogy

Key movement

- Growing from m to $m+1$.
- Fraction $\frac{1}{m+1}$ of keys stays. Rest moves.
- So $(n - \frac{n}{m+1}) = \frac{nm}{m+1} \sim n$. (Essentially all keys move!)
- Even worse than doubling!



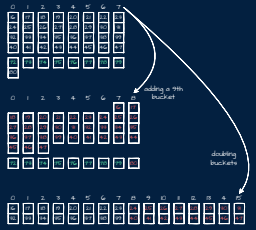
Let's make this a bit more precise. When we go from m to $m+1$ buckets, the only keys that stay (map to the same index) are those that map to $[k\ell, k\ell + m[$, where $\ell = \mathrm{LCM}(m, m+1)$ is the least common multiple of m and $m+1$ (and $k \in \mathbb{Z}$). Since m and $m+1$ are relatively prime, the least common multiple is $m(m+1) = m^2 + m$. So, as the number of buckets m grows, the least common multiple grows quadratically, which means the spacing between these "islands of stability" grows quadratically. At the same time, the "size" of each island grows linearly. Hence the number of stable keys (that stay) goes asymptotically to 0.

Another way of putting this, a bit more precisely, is that every $m(m+1)$ keys, there is a sequence of m stable keys. So, out of n keys, a fraction $\frac{m}{m(m+1)} = \frac{1}{m+1}$ is stable, which means $\frac{n}{m+1}$ keys stay. The remaining $n - \frac{n}{m+1} = \frac{n(m+1)}{m+1} - \frac{n}{m+1} = \frac{nm}{m+1}$ move. But note that $\frac{nm}{m+1} \sim n$.

So **essentially all keys move**! This is even worse than doubling!

Mod hashing bad!

- Normally, doubling works fine.
  - Copying pointers in memory is super fast.
  - Can't do that over the network.
  - "Distributing" the data structure encounters new constraints.
- Growing by one is even worse, but it's the common case in the distributed (especially decentralized) case!
- Also: Need to know the number of buckets/nodes to be able to find keys!
- Need some fundamentally different way to partition keys!



The bottom line is that mod hashing is bad and won't work in this context! For an in-memory data structure, it works just fine, because copying pointers in memory is really fast. But it's too slow if we have to do it over the network. So trying to "distribute" the hash table data structure has revealed some new constraints.

Doubling the number of buckets didn't work. But growing by one is even worse, and that's precisely the common case we have to support. In the decentralized case, we'll have nodes coming and going, joining and leaving the system seemingly at random. But even in the centralized case, we would like to add (and remove) only a few nodes at a time to adapt to changes in workload.

So we have to figure out some fundamentally different way to partition keys. Maybe there are some other, more suitable ways of implementing a dictionary/map, other than a hashtable?

Binary trees

Simple Key-Value Store (KVS)

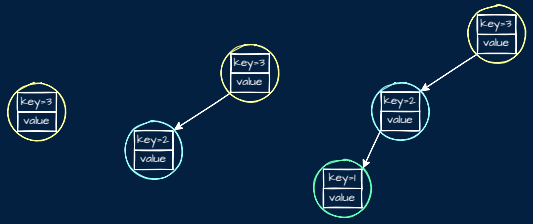
- Sorted array had $\Theta(\log_2 n)$ `get` but $\Theta(n)$ `put`.
- With a (balanced) binary tree, we can have $\Theta(\log_2 n)$ `get` and `put` (worst case).
- Need to keep it balanced. Maybe an AVL tree.

With a sorted array, we had $\Theta(\log_2 n)$ `get` but $\Theta(n)$ `put` (both worst case). With a balanced binary tree, both `get` and `put` are $\Theta(\log_2 n)$ (worst case). Of course, we'll need to keep it balanced!

Let's consider an AVL tree.

AVL trees

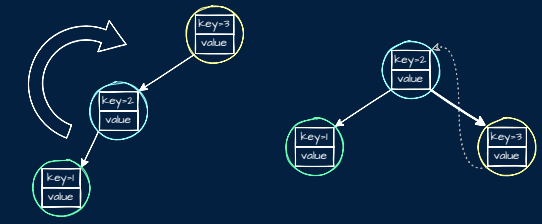
Simple Key-Value Store (KVS)



Here is a sequence of 3 insertions. But now the tree has become unbalanced...

AVL trees

Distributed KVS

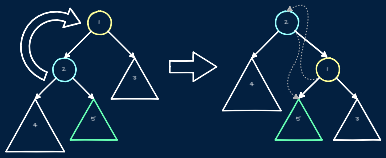


...so we need to do a rotation to restore balance.

AVL trees

Distributed KVS

- Technically only need to swap "pointers".
- Requires a lot of coordination!
- What if we keep the location of nodes tied to machines and move keys?



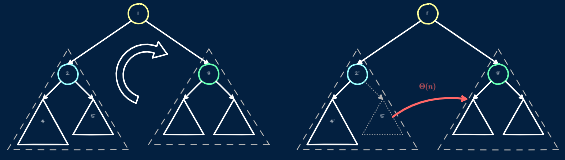
We technically only need to swap a few pointers, which in our distributed case just means swapping some node addresses.

However, this requires a lot of coordination. We wouldn't want two (or more) different, concurrent insertions to cause two (or more) simultaneous rotations. Then we might lose part of the tree due to what is essentially a data race. But locking the whole "data structure" is not really feasible. How do the different nodes in the tree know about what other nodes are doing? They would first have to agree with each other on what update to make. The fine granularity (with one node per KV pair) makes it difficult to maintain a coherent picture of the whole.

What if, rather than move nodes around (by swapping pointers), we keep the structure the same and just move the keys between nodes to achieve the same rotation?

AVL trees

Distributed KVS

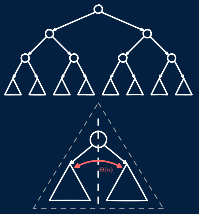


So, let's keep the positions of the nodes fixed and just move keys. But now we find that we need to move $\Theta(n)$ keys again!

Problems

BST / AVL

- Storing a single key-value pair at each node!
- Would need a ridiculous number of nodes!
- $\Theta(\log_2 n)$ sounds good for in-memory. But too many hops in the network!
- Even if we let nodes be roots of sub-trees
  - Storing a single key-value pair at each “internal” node!?
  - Always need to start at the root node. (Cache?)
  - Still have the same problem with $\Theta(n)$ key movement!



There are also some other problems. Each node only stores a single KV pair, which means we’ll need n nodes! Which also means we need to traverse the tree by going from one node to another over the network.

Now, we could let a node be the root of a whole sub-tree. But then do we still have a single KV pair at each internal node until we get sufficiently deep where each node is responsible for the whole rest of the tree?

We also have the same problem that we had with the hash table, which is that we still start from a single root node. Although, caching might technically be more feasible in this case.

Still, the big one is linear key movement!

Problems

BST / AVL

- D-ary instead of binary. (Larger fanout, shallower tree.)
- Store more data per node.
- Even better: Store data only in leaves.
- Sound familiar?

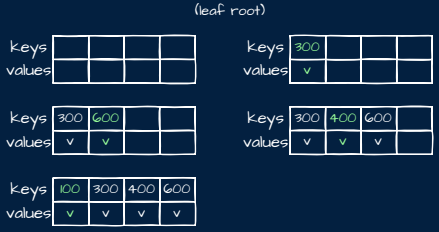
Let’s try to address some of these problems directly.

If we use a D-ary tree instead of a binary tree, we get a larger fanout and a shallower tree. And we’re storing more data per node, so we need fewer nodes. So we only need $\Theta(\log_D(n/D))$ hops instead of $\Theta(\log_2 n)$.

Even better would be to store data only in the leaves rather than spread out in the whole tree. At this point, this should start to sound familiar. You may have come across B+ trees in the context of databases.

B+ trees

Distributed KVS

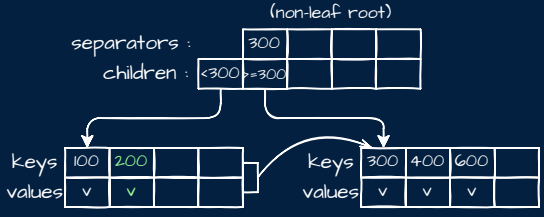


This illustration shows a single node that starts out empty and then has 4 key-value pairs added to it.

Initially, we only have one node, which is both the root and a leaf. And it can store up to D values before it becomes full and needs to be split.

B+ trees

Distributed KVS

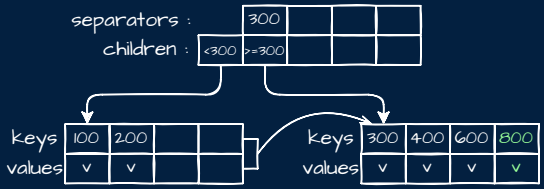


But once it becomes full, we need to split it up into two leaf nodes and create a new (non-leaf) root. The root does not store any values. Only leaves store values. The root stores (up to) D separators and (up to) $D + 1$ references to child nodes. In this case there are two children and one separator value. Everything less than 300 is in the first child node, and everything greater than or equal to 300 is in the second child node.

Each leaf also has a pointer to its successor (if any), i.e. its rightmost sibling. This provides some resistance against stale child pointers: if an interval has been split, the search can continue in the next leaf.

B+ trees

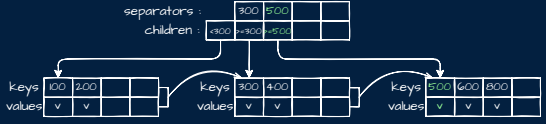
Distributed KVS



Let's fill up another leaf node...

B+ trees

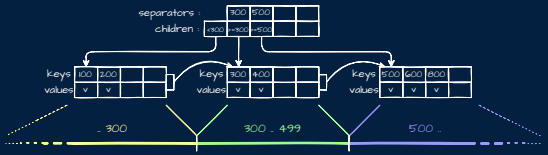
Distributed KVS



Now it, too, is split, dividing the keys between the new left and right nodes. The root now has a third child, each containing keys in three intervals, and so the root has two separators.

B+ trees

Distributed KVS



The fact that each leaf represents an interval is important!

B+ trees

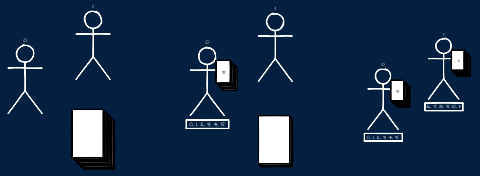
Distributed KVS



Each interval can be split (roughly) in half, and only half of the keys in the interval need to move from one node to another!

Revisiting the card dealing analogy

B+ trees

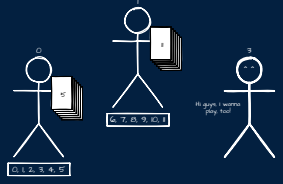


Let's revisit the card dealing analogy again. Since we're partitioning keys in a different way, that means we are dealing the cards in a different way in the analogy.

Instead of cycling through the players repeatedly and giving each player one card at a time until we've dealt the whole deck of n cards, suppose we just give each of the m players n/m of the cards. So, if there are $m = 2$ players, we give the first player the first half of the deck, i.e. cards $0, 1, 2, ..., n/2$, and the second player the other half, i.e. $n/2 + 1, n/2 + 2, ..., n - 1$.

Revisiting the card dealing analogy

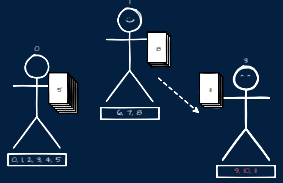
B+ trees



Now suppose a new player joins.

Revisiting the card dealing analogy

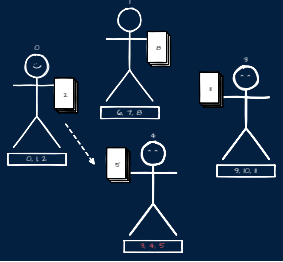
B+ trees



We just need one of the players to give half of its cards to the new player.

Revisiting the card dealing analogy

B+ trees



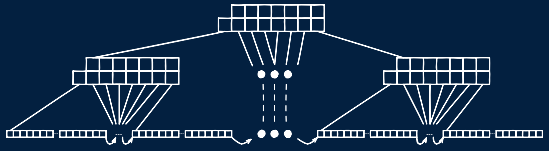
And the same thing happens if a fourth player joins. Note that we are only moving some of the cards one of the players has.

As the number of players increases, each player has a smaller and smaller fraction of all cards. And then we move some fraction of what one single player has. This means that, as the number of players increases, **the fraction of all *n* cards that must move gets smaller and smaller**!

When we used mod hashing, we got exactly the opposite result, because the fraction got bigger and bigger as we added more players!

B+ trees

Distributed KVS

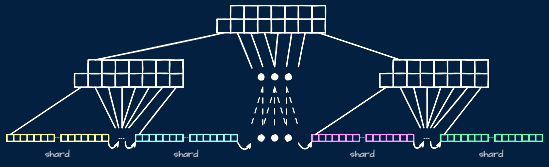


Let's get back to the B+ trees, and let's imagine a really large tree with lots of data.

The illustration hides some of the nodes because the tree would be extremely wide otherwise. Each leaf stores a whole range of keys.

B+ trees

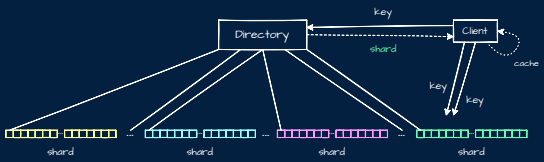
Distributed KVS



Since KV pairs are only stored at leaf nodes, and each leaf stores keys in a particular interval, we could treat each leaf as a separate **shard**.

B+ trees

Distributed KVS



Since shards store **ranges** of keys, they can be split without massive key movements.

And then we could collapse the first few levels of the tree into a single directory service that just maps keys to shards and lets us find the right shard. In fact, because the key-to-shard mapping is now much more stable, we can let the client cache the shard so it can talk directly to the right shard, thus bypassing the directory.

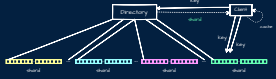
Note that we can also make the directory service and each shard be replicated for fault-tolerance.

This kind of design is used by real systems.

What next?

Distributed KVS

- Centralized.
- No bottleneck root (caching).
- Replication prevents single point of (technical) failure.
- But still centrally administered.
- Someone “in charge”.



However this is still a centralized system. While we have gotten rid of the single bottleneck at the root thanks to caching, and while we don’t have a single point of failure in the technical sense (because we can use replication to achieve fault-tolerance), we need different nodes to play different roles. Who decides who should run the directory service versus who should run a leaf shard, and which one? In other words, this design still requires that someone is “in charge”.

New requirement: Decentralized

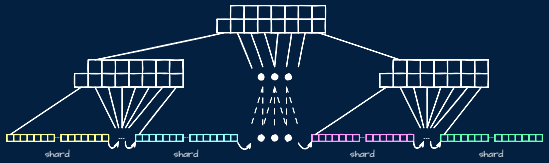
- Not just distributed, but also decentralized.
- Nobody is in charge.
- All nodes equivalent.
- No “special” (central) node.
  - No bottleneck.
  - No single point of failure. (Technical or otherwise.)
- Nodes come and go spontaneously (typically one by one). (Churn)

So, now let’s consider a system that is not only distributed but also decentralized. We would like all nodes to be equivalent so there are no “special” nodes such as directory versus leaf/shard. While it is technically possible to have special roles and dynamically assign different nodes to playing different roles, that makes the solution much more complex. And we would like to keep it simple, at least for now.

Since nobody is in charge, nodes just come and go spontaneously. That means we have to handle churn.

When a new node joins, it has to become part of the system and start contributing to its functionality. And when it leaves, the remaining nodes should be able to keep operating so everything still works. This immediately implies some redundancy, or else some functionality (or data) would be lost when a node leaves.

Designing a decentralized DHT



Take another look at this B+ tree.

Designing a decentralized DHT



Notice something about the leaves?

Designing a decentralized DHT



It's a linked list!

And linked lists actually have a couple of nice properties. First, it's a recursively defined data structure. That is, each linked list node is itself a linked list (it's the head of a list, which is the head of a list, etc.). If we want all nodes to be equivalent and interchangeable, that's a useful property.

We also know we can easily insert a new node anywhere in the list.

Linked lists?

Designing a decentralized DHT

- Let each leaf be a node storing keys in some range.
- We want to be able to reach any node from any node.
- Unfortunately can only go “to the right”.
- How do we get to the first node?
- Make it circular! Wrap around back to 0.

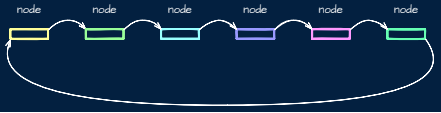


Let each node store keys in some range, just like the leaves in the B+ tree. But we can only follow links “forward”, and we don’t want to have to keep track of a global *first* node. So how can we get from any node to any other node? One way would be to make it circular by letting the last node point to the first node. In other words, we wrap back around to 0.

Linked lists?

Designing a decentralized DHT

- Let each leaf be a node storing keys in some range.
- We want to be able to reach any node from any node.
- Unfortunately can only go “to the right”.
- How do we get to the first node?
- Make it circular! Wrap around back to 0.
- Suggests finite keys. Collisions?
- Use (for example) `SHA-256`: 256 bits, probability of collisions negligible.
- But $\Theta(N)$ to walk the list??



Then “first” and “last” are no longer important, and any node is equivalent to any other. If we know about one of the nodes, we can get to any other node, and we can find any key.

Now, it’s not strictly necessary to use a finite key space, but it certainly makes things a lot simpler. But then what about collisions? Just use a cryptographic hash function, like SHA-256. In other words, we’ll have something like $B = 256$ bits, which means there are 2^{256} possible keys, an astronomically large number. By the pigeonhole principle, collisions are necessarily still possible, but cryptographic hash functions make them difficult to produce on purpose, and so the probability of collisions is negligible.

However, if we have N nodes (both “linked list” node and machines) in the system, we may have to take $\Theta(N)$ steps to find the right node and the key. That’s not acceptable. Can we somehow do better?

Skiplist

Designing a decentralized DHT



What about a *skiplist*? It's like a linked list where each node may have additional references to later nodes, like shortcuts.

Here we have a regular linked list. Let's add some shortcuts to it.

Skiplist

Designing a decentralized DHT



For example, here we have one extra shortcut that allows us to cut the linked list in half.

Skiplist

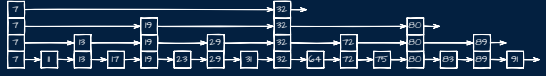
Designing a decentralized DHT



And if we keep adding more shortcuts like this...

Skiplist

Designing a decentralized DHT



...we end up with a data structure that's kind of a linked list but that also allows $\Theta(\log N)$ insert and delete.

Skiplist

Designing a decentralized DHT



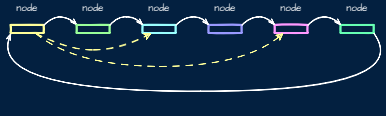
Here's an example lookup that illustrates how these shortcuts can be used. Note that we had to take 6 steps rather than the 13 steps that would have been required by a purely linear scan in a regular linked list.

Now, I should point out that i have hand-crafted this example to be the ideal skiplist, which would normally insert shortcuts probabilistically. But then we would still end up with $\Theta(\log N)$ in expectation.

Now let's consider how we could combine this with the circularly linked nodes.

Skiplist

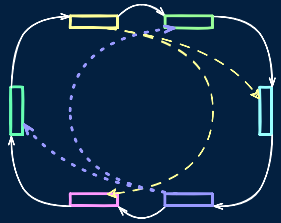
Designing a decentralized DHT



Here we have some of these skip links added to two of the nodes. As you can see, each node always has a pointer to its immediate successor (at distance $2^0 = 1$). But then it also has shortcuts to nodes at distance $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, and so on.

Skiplist

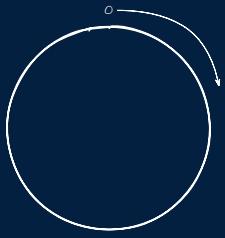
Designing a decentralized DHT



Let me just rearrange the nodes a bit so they look slightly more circular.

Skiplist

Designing a decentralized DHT

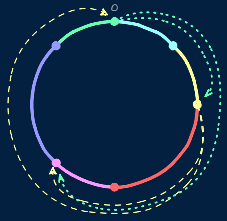


Now, we only have some relatively small number of nodes $N \ll 2^B$ in the system. But we can represent the whole key/ID space as a continuous circle, going from 0 to $2^B - 1$ and wrapping back around to 0.

What we need to do is map ranges of this key space to the N nodes we do have in the system.

Skiplist

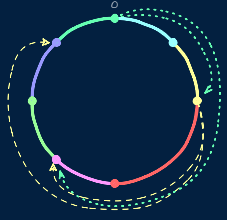
Designing a decentralized DHT



Each dot on the circle represents one node. It is responsible for all keys ***after*** *its immediate predecessor* up to and including its own ID. That is, each key maps to the closest successor node (the next higher node ID, which may wrap around past 0).

Skiplist

Designing a decentralized DHT



Here a new node has joined, splitting one of the ranges and causing some of the shortcuts to be updated.

Chord

Designing a decentralized DHT

This is essentially the Chord [15, 14] DHT.

- A key k maps to the node that is its immediate successor. (First node encountered going clockwise around the circle.)
- "Finger table" stores $\Theta(\log N)$ peers and allows $\Theta(\log N)$ lookup.
- Splitting an interval ($N \to N+1$) moves a fraction $\Theta\left(\frac{1}{N+1}\right)$ of keys!
- This is the important property of **consistent hashing**.
- The key-to-bucket mapping stays consistent. (Unlike the "inconsistent" modulo hashing.)

We have pretty much reinvented the Chord [15, 14] DHT. Note that we have only covered the essential, high-level concepts behind how the Chord DHT works. Of course, if we wanted to implement Chord, we would need a lot more detail, such as how to keep the routing table (or "finger table") up-to-date and how to redistribute keys as nodes join and leave the network.

One concept that is often associated with Chord is *consistent hashing*. The important property is that adding a new node splits an interval, and only fraction $\Theta\left(\frac{1}{N+1}\right)$ of the keys need to move between nodes. And that is not specific to Chord. (We saw the same behavior with B+ trees, for example.)

Chord

Designing a decentralized DHT

- Intervals are split wherever a node ID happens to land, not in the middle.
- Even if node IDs are uniformly distributed, interval sizes are not!
- Fix: Virtual nodes.
- What if a node crashes? Must hand over keys!
- Replicate on k successor nodes. Then we can just jump to the next one.

A cryptographic hash function produces uniformly distributed outputs (keys and IDs). However, uniformly distributed node IDs still result in exponentially distributed gaps (which correspond to the intervals nodes are responsible for). And assuming uniformly distributed keys means that the density of keys is the same everywhere. So bigger gaps translate to more keys.

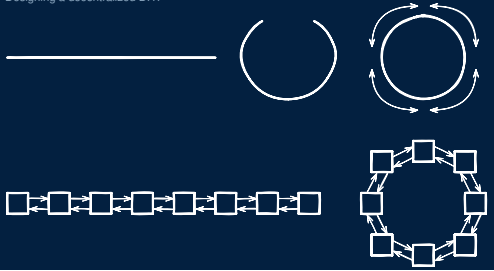
Nodes will have some average interval size (or arc length) and therefore key count. However, the maximum is $\Theta\left(\frac{\log N}{N}\right)$, which is much larger than the mean of $\frac{1}{N}$: $\max/\text{mean} \in \Theta(\log N)$. Chord's solution to this is *virtual nodes*: each physical machine participates multiple times and corresponds to multiple nodes in the Chord network. If there are M machines, each needs to run $V \in \Theta(\log M)$ virtual nodes to make the ratio constant rather than logarithmic, and so load balance is achieved. You can read more about this in [8].

Finally, we need some redundancy so that keys are not lost when a node suddenly goes offline. A simple solution is to replicate each key on k successor nodes. If the node that was supposed to have the key is down, we can just jump to its successor, which is also supposed to have a copy, etc.

Some day, I may record a video describing it in more detail. Until then, you can read the original paper(s) [15, 14]. You'll also easily find a lot of info online if you do some Googling.

CAN

Designing a decentralized DHT



Instead of dividing up the interval from 0 to $2^B - 1$ into N segments corresponding to the N nodes and arranging them as a circular linked list, let's make it a circular *doubly linked list*. Making it a doubly linked list allows any node to reach any other node, but we need to make it circular to avoid making the head or tail special cases. Now all nodes are equivalent. We still have the problem that getting from any one node to any other takes $\Theta(N)$ steps.

CAN

Designing a decentralized DHT



- A circle is just a 1-dimensional torus.
- We can generalize to $d \geq 2$ dimensions.
- Can wrap around along all d dimensions.
- Each node has $2d$ neighbors (successor/predecessor).

But now imagine that, instead of a one-dimensional interval, we have a two-dimensional surface. We can cut it up into a grid, where each node is responsible for a 2D area (a *"zone"*) rather than a 1D interval, and now each node has a successor and predecessor along each axis. If we imagine rolling up the 2D surface into a tube and connecting both ends along one axis, and then curving the tube to connect the two ends along the other axis, we end up with a 2D torus and circular wrapping around along both dimensions.

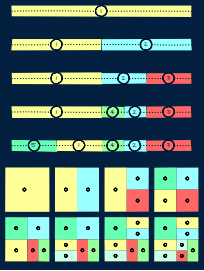
We can extend this to as many dimensions as we would like, but it quickly gets difficult to visualize or even imagine.

CAN

Designing a decentralized DHT

- Imagine a k-d tree.
- But instead of points, leaves are “zones”.
- Each zone has an owner.
- When a node joins, the zone splits along one dimension.
- Keys in one half get transferred to the owner of the new zone.
- Again $\Theta\left(\frac{1}{N+1}\right)$ of keys move.

This is essentially CAN [11] (the Content-Addressable Network).



A binary search tree allows us to efficiently find points arranged along a single dimension. If we have points in 2 or more dimensions, we can use a k-d tree, which generalizes a binary search tree from 1 to k dimensions.

CAN (the Content-Addressable Network) works very similarly, but instead of individual points, the leaves are “zones” (containing multiple KV pairs). (Another important difference is that the k-d tree splits keys into equal halves by finding the median, while CAN splits zones by volume, regardless of the number of keys in each half. By the assumption that keys are uniformly distributed, the density is the same, and therefore the number of keys is expected to be the same since the volumes are the same.)

Each zone has an owner that is responsible for all the keys in that zone. When a node joins, it picks some point (i.e. coordinates) in the d-dimensional space as its ID. This point belongs to some zone, which is split in half along one dimension, creating a new zone. Ownership of the half that contains the ID is transferred to the new node, along with all the keys in that half.

Because we’re again splitting intervals, we again have $\Theta\left(\frac{1}{N+1}\right)$ key movement.

CAN

Designing a decentralized DHT

Lookup/routing:

- Only keep track of $\Theta(d)$ neighbors (constant).
- When all zones are equal, each node has exactly $2d$ neighbors.
- Which of my neighbors has its zone closest to the target?
- Forward to it.
- Lookup/routing in $\Theta\left(d\sqrt[d]{N}\right)$ steps.

CAN uses *expected* constant-size routing tables because each node only keeps track of a few predecessors and successors for each of the d dimensions. For example, left/right, up/down, etc. In the special case of evenly split (equal-sized) zones, there are exactly two neighbors per dimension. While there is technically no upper bound on the number of neighbors, large values are increasingly unlikely, and so the number of neighbors is $\Theta(d)$ in expectation.

Zone owners forward lookups greedily to the next neighbor along one of the d dimensions. That is, it checks which of its neighbors owns the zone that is closest to the target key, and then forwards to that neighbor.

CAN

Designing a decentralized DHT

- Chord improved lookup performance by remembering more state ($\Theta(\log N)$ finger table entries, shortcuts).
- CAN allows the trade-off between state size and hop count to be tuned by setting d based on the expected network size.
- Busy zones (containing popular keys) can be offloaded by caching *"around"* them.

Chord uses a finger table keeping track of $\Theta(\log N)$ "neighbors" as shortcuts to avoid a linear scan around the "linked list" and achieve lookup in $\Theta(\log N)$ steps. It's exactly the same trade-off the skiplist made as well. While there is potentially a table entry for each of the B bits, only $\Theta(\log N)$ of those entries would actually be distinct nodes, which is what ultimately matters.

CAN does not allow us to circumvent that trade-off, but it makes it tunable and has a number of nice properties that follow from the geometry of the key space. For a specific network size, we can choose d such that CAN uses the same amount of state and needs the same number of steps during lookup/routing. But CAN has greater path diversity, which allows lower latency and resilience. Chord has a single appropriate entry in its finger table, i.e. a single greedy path. If it doesn't use that one, it needs more hops. CAN can have multiple equivalent choices and can prefer lower-latency neighbors, i.e. RTT-weighted routing.

CAN

Designing a decentralized DHT

Cool extensions (for redundancy but also improved lookup latency):

- Zone overloading, i.e. joint zone ownership. (Replication.)
- Multiple hash functions. (Each item has multiple keys.)
- Multiple realities. (Similar to virtual nodes, but adding replication.)

As usual, we can improve fault-tolerance and performance by redundancy, i.e. by sacrificing memory/storage.

CAN supports multiple interesting ways to achieve different kinds of redundancy. This redundancy ensures that keys are not lost, but it can also improve lookup latency and reliability.

Zone overloading means that each zone has more than one owner, which means that the keys in the zone are stored by more than one node. If one of them goes down, the keys in that zone are not lost, because other owners remain. In addition, latency can be reduced because only one of the owners needs to respond.

By using k different hash functions, each value is mapped to by k different keys. This means each value is stored in up to k different zones and therefore by up to k different nodes (or more when combined with zone overloading). If one of the k keys is lost, it is likely that the value can still be found using one of the other keys. One of the k keys may be closer than the others, which means the value can be looked up using the closest key, improving lookup latency.

Finally, multiple realities means that there are several parallel instances of the system. Each node has a different ID in each reality, and so it stores different keys in each reality. It also has different neighbors in each reality. But routing/lookup requires only a match in one of the realities. A node can forward a request to the closest neighbor in any reality, and if a neighbor is down or a key is lost in one reality, another path can be taken in a different reality.

Consistent Hashing

- Purpose/advantage: when adding/removing buckets, only a small fraction of keys map to a different bucket.
- Adding bucket $N+1$ (have N buckets): expect $\frac{1}{N+1}$ keys to move.
- Removing bucket N (have N buckets): expect $\frac{1}{N}$ keys to move.
  - For example, going from 1 to 2 buckets: about half of the keys move to the new bucket.
  - For example, going from 2 to 1 bucket: about half of the keys move from the deleted bucket.
- Note that the fraction shrinks! (Depends on N; not constant!)

Just a quick reminder/recap of the interesting property of consistent hashing, to be contrasted with the "inconsistent" mod hashing.

DHTs and Overlay Networks

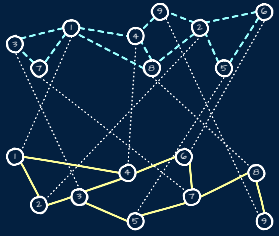
- An overlay network is a logical network "overlayed" on top of an existing network.
- DHTs are overlay networks with built-in functionality for storing key-value pairs.
- Nodes have addresses/IDs in the overlay network.
- We can (conceptually) send them messages using their IDs. (Compare with DNS.)
- Some distance metric* is used for (greedy) routing/lookup. Defines a topology.
- Could use the overlay network for other things than KVS.

* More accurately: a notion of "closeness". (Not necessarily a mathematical metric.)

We have seen two ways to look at DHTs: as literal ***Distributed** Hash Tables* or as a key-value store, i.e. a specialized database. Another way to look at them is as an **overlay network** with data storage functionality added.

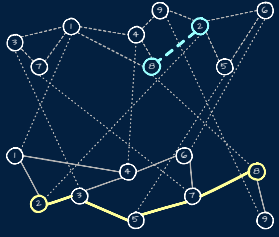
An overlay network can also be used for other things, other than storing key-value pairs. In a DHT, when we want to store a new key-value pair, it has to get to some suitable node. So the network either has to route messages to that node (recursively), or we have to find the node (iteratively) so we can talk to it ourselves. So, if we just wanted to send it a message, we could do that. The overlay network can add additional properties [2, 13, 3, 1] that the underlying network doesn't have. This is analogous to how TCP adds functionality on top of IP.

DHTs and Overlay Networks



The same nodes exist in two networks, with the overlay on top. We can map each node in the overlay network to a node in the underlying network.

DHTs and Overlay Networks



The distance between two nodes in the overlay network may be very different (much longer or much shorter) compared to the distance between the nodes in the underlying network (which is typically roughly similar to physical distance).

Properties of Overlay Networks (and DHTs)

Properties:

- Lookup performance
- Maintenance overhead
- Load balance (mostly DHT)
- Congestion
- Stretch
- Fault-tolerance
- Routing table size versus lookup (degree optimality)

Example DHTs

- Pastry [12]
- Tapestry [17, 16]
- CAN [11]
- Chord [15, 14]
- Kademlia [10, 6]
- Viceroy [9]
- Koorde [7]
- Broose [5]
- D2B [4]
- LAND [1]
- etc.

We have already talked about some of these properties, like lookup performance, load balance, and fault-tolerance. We also just saw an example of *stretch*: the ratio by which the overlay stretches physical distances, i.e. how much longer paths between two nodes are in the overlay compared to paths in the underlying network.

It is possible to achieve both $\Theta(1)$ routing tables and $\Theta(\log N)$ lookup, which is *"degree-optimal"*. A degree-optimal DHT/overlay with $\Theta(\log N)$ routing table can instead achieve $\Theta\left(\frac{\log N}{\log \log N}\right)$ lookup/routing. Neither Chord nor CAN (nor Kademlia) are degree-optimal. While CAN has constant-size routing tables, its lookup performance is asymptotically worse than $\Theta(\log N)$.

There are many more DHTs with different properties and trade-offs. I recommend reading more about them and trying to understand their relative advantages and disadvantages. It's a fun exercise.

References I

[1] Ittai Abraham, Dahlia Malkhi, and Oren Dobzinski. "LAND: stretch (1+ epsilon) locality-aware networks for DHTs.". In: *SODA*. 2004, pp. 550–559.

[2] Ittai Abraham et al. "A generic scheme for building overlay networks in adversarial scenarios". In: *Proceedings International Parallel and Distributed Processing Symposium*. IEEE. 2003, 9–pp.

[3] Mayur Datar. "Butterflies and peer-to-peer networks". In: *Algorithms—ESA 2002: 10th Annual European Symposium Rome, Italy, September 17–21, 2002 Proceedings 10*. Springer. 2002, pp. 310–322.

[4] Pierre Fraigniaud and Philippe Gauron. "D2B: A de Bruijn based content-addressable network". In: *Theoretical Computer Science* 355.1 (2006), pp. 65–79.

References II

[5] A-T Gai and Laurent Viennot. "Broose: A practical distributed hashtable based on the de-bruijn topology". In: *Proceedings. Fourth International Conference on Peer-to-Peer Computing, 2004. Proceedings.* IEEE. 2004, pp. 167–174.

[6] Bernhard Heep. "R/Kademlia: Recursive and topology-aware overlay routing". In: *2010 Australasian Telecommunication Networks and Applications Conference*. IEEE. 2010, pp. 102–107.

[7] M Frans Kaashoek and David R Karger. "Koorde: A simple degree-optimal distributed hash table". In: *Peer-to-Peer Systems II: Second International Workshop, IPTPS 2003, Berkeley, CA, USA, February 21-22, 2003. Revised Papers 2*. Springer. 2003, pp. 98–107.

References III

[8] David Karger et al. “Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web”. In: *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*. 1997, pp. 654–663.

[9] Dahlia Malkhi, Moni Naor, and David Ratajczak. “Viceroy: A scalable and dynamic emulation of the butterfly”. In: *Proceedings of the twenty-first annual symposium on Principles of distributed computing*. 2002, pp. 183–192.

[10] Petar Maymounkov and David Mazieres. “Kademlia: A peer-to-peer information system based on the xor metric”. In: *International Workshop on Peer-to-Peer Systems*. Springer. 2002, pp. 53–65.

[11] Sylvia Ratnasamy et al. “A scalable content-addressable network”. In: *Proceedings of the 2001 conference on Applications, technologies, architectures, and protocols for computer communications*. 2001, pp. 161–172.

References IV

[12] Antony Rowstron and Peter Druschel. “Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems”. In: *Middleware 2001: IFIP/ACM International Conference on Distributed Systems Platforms Heidelberg, Germany, November 12–16, 2001 Proceedings 2*. Springer. 2001, pp. 329–350.

[13] Jared Saia et al. “Dynamically fault-tolerant content addressable networks”. In: *Peer-to-Peer Systems: First InternationalWorkshop, IPTPS 2002 Cambridge, MA, USA, March 7–8, 2002 Revised Papers 1*. Springer. 2002, pp. 270–279.

[14] Ion Stoica et al. “Chord: a scalable peer-to-peer lookup protocol for internet applications”. In: *IEEE/ACM Transactions on networking* 11.1 (2003), pp. 17–32.

[15] Ion Stoica et al. “Chord: A scalable peer-to-peer lookup service for internet applications”. In: *ACM SIGCOMM computer communication review* 31.4 (2001), pp. 149–160.

References V

[16] Ben Y Zhao et al. “Tapestry: A resilient global-scale overlay for service deployment”. In: *IEEE Journal on selected areas in communications* 22.1 (2004), pp. 41–53.

[17] Ben Yanbin Zhao, John Kubiatowicz, Anthony D Joseph, et al. “Tapestry: An infrastructure for fault-tolerant wide-area location and routing”. In: (2001).